

DAA HACKATHON PROJECT REPORT

DISASTER RELIEF VEHICLE LOADING

*Choosing relief items for vehicles with limited capacity, giving priority to essential goods
and vulnerable communities*

Subject	Design and Analysis of Algorithms
Problem Statement	Problem Statement 88 - Disaster Relief Vehicle Loading
Core Algorithms	Greedy Algorithm and Priority Queue
Suggested Technologies	Python, Flask, JavaScript
Project Designation	DAA Hackathon / Academic Mini-Project
Team Members	R KISHOR - 25MVCSMR0075 A PRADEEP - 25MVCSMR0005 T SANDEEP - 25MVCSMR0083 CH SRIDHAR - 25MVCSMR0018

Submitted in partial fulfilment of the requirements of the course
Design and Analysis of Algorithms

Abstract

After floods, earthquakes or cyclones, relief agencies hold far more goods than their vehicles can carry. Every vehicle has a limited load capacity, so someone must decide which relief items go on which vehicle and which items must wait for the next trip. A careless choice can send a truck full of low-urgency goods to a safe area while medicines and drinking water remain in the warehouse. Choosing the load is therefore an algorithmic decision problem, closely related to the knapsack problem.

This project, **Disaster Relief Vehicle Loading** (Problem Statement 88), simulates the loading of relief items onto several vehicles. Every item has a weight, an **essentiality** score for the type of goods and a **vulnerability** score for the community that will receive it. These are combined into one integer **priority score** $P = 2e + v$, so essential goods count twice as much as community vulnerability. Every vehicle has a capacity, a current load and a free capacity, and the system explains each loading decision to the user.

Two classical techniques are used. A **Priority Queue**, implemented as a binary heap, stores the waiting items so that the item with the highest priority score is always extracted first in $O(\log n)$ time. A **Greedy Algorithm** then loads that item into the feasible vehicle that leaves the **smallest unused space** (best fit), and an item that fits no vehicle is deferred. The choice is locally best and is never revised, which keeps the method fast and easy to explain.

The total running time is $O(n \log n + n \cdot V)$ for n items and V vehicles, and the space requirement is $O(n + V)$. The report states honestly that choosing items for a limited capacity is a 0/1 knapsack-type problem, so the greedy method is a heuristic without a guarantee of the best possible load, and it illustrates this with a counter-example.

The prototype is planned with Python and Flask for the algorithm and API layers and HTML, CSS and JavaScript for the dashboard. Sample, simulation-based scenarios show that priority-first loading places essential goods and vulnerable-community items on the vehicles before routine goods and loads more of them than simple arrival-order loading on the demonstration workload. The expected outcome is an understandable, algorithm-centric prototype that demonstrates practical use of the Greedy Algorithm and the Priority Queue.

Keywords: Disaster relief, vehicle loading, limited capacity, essential goods, vulnerable communities, Greedy Algorithm, Priority Queue, binary heap, best fit, knapsack.

1. Introduction

1.1 Background of Disaster Relief Distribution

In a disaster, relief supplies such as water, medicines, food, blankets and tents arrive at a central depot and must be moved to affected communities by trucks, vans and other vehicles. The supply is usually larger than the capacity of the vehicles available on a given day, and the needs of the communities differ in urgency. The person or program that decides what goes on each vehicle is performing the **vehicle-loading** task studied in this project.

1.2 Multiple Vehicles with Limited Capacity

Each vehicle can carry only a fixed load, for example 100 kg for a van and 200 kg for a truck. Some vehicles may already be partly loaded or may be unavailable. For every item the loader must decide on which vehicle it is placed, if any, while considering what space remains for the items still waiting. The state of every vehicle changes with each item loaded, so decisions must be taken one after the other with the current state in view.

1.3 Problems Caused by Poor Loading Decisions

- **Essential goods left behind:** drinking water or medicines wait while less urgent goods are carried.
- **Vulnerable communities served late:** children, elderly people or isolated villages receive help last.
- **Wasted capacity:** vehicles leave partly empty because the remaining space is fragmented.
- **Arbitrary order:** loading in arrival order ignores urgency and is hard to justify afterwards.
- **No explanation:** nobody can say why an item was or was not loaded.

1.4 Importance of Priority-Based Loading

Limited vehicle capacity means that not everything can be sent at once. Priority-based loading makes sure that the most valuable use is made of every kilogram: essential goods and the needs of vulnerable communities come first, and the space on every vehicle is used well.

1.5 Motivation for the Proposed System

The motivation is to turn this decision into a small, transparent simulator in which every step can be inspected: which item was taken next, which vehicles could take it, which vehicle was chosen and why, and which items were left behind. This makes the project a suitable vehicle for explaining and analysing algorithms in a viva or a presentation.

1.6 Role of Algorithms in Loading Decisions

Two algorithmic ideas solve two sub-problems. The **Priority Queue** decides **which item is considered next**, and the **Greedy Algorithm** decides **which vehicle receives it**. Both are simple and efficient, and have clear complexity bounds, so the loading procedure is fast and can be analysed rigorously.

1.7 Existing Approach and Proposed Approach

Basic / Manual Loading

The simplest practice is to load items in the order in which they arrive at the depot (**first-come first-served**) or according to the judgement of the loading crew. This needs no computation and is easy to carry out.

Limitations of Simple Loading

- It ignores the **essentiality** of the goods, so vital items may be left for the next trip.
- It ignores the **vulnerability of the destination community**.
- It ignores **vehicle capacity**, so large items may block space that several smaller, more urgent items could use.
- It gives **no explanation** of why an item was loaded or left.

Need for Algorithmic Loading

A method that uses the properties of every item and the free capacity of every vehicle can avoid these problems. Because the decision is made for many items, it must also be cheap: examining every vehicle once per item is acceptable, whereas searching all possible loads is not. This is the setting in which a greedy rule combined with a priority queue is attractive.

Proposed Disaster Relief Vehicle Loading System

The proposed system keeps a configurable set of simulated vehicles and a priority queue of waiting relief items. It repeatedly extracts the highest-priority item, finds the vehicles that can still carry it, loads it onto the one that leaves the least unused space, updates the vehicle state and records the decision and the metrics. A dashboard displays the decision and the reason.

High-Level Workflow

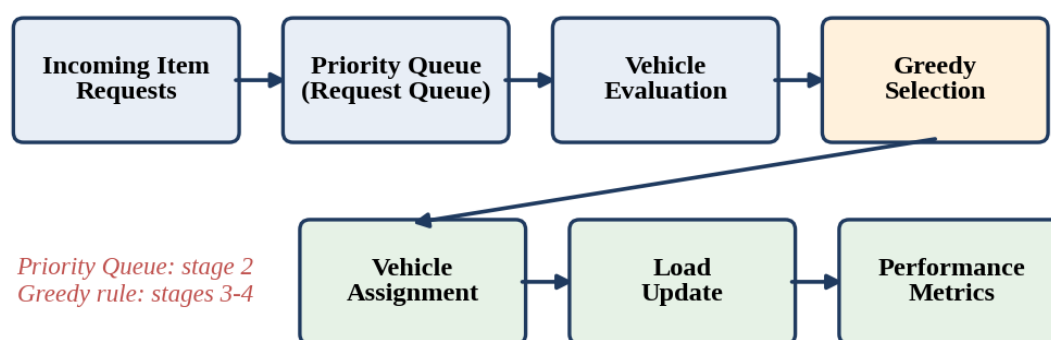


Figure 1: Disaster Relief Item Loading Workflow

Item requests enter a priority queue. Each extracted item triggers an evaluation of all vehicles, the greedy rule selects one, and the assignment updates the load and the performance metrics shown to the user.

2. Problem Statement

2.1 Official Problem Statement (Problem Statement 88)

“Choose relief items for vehicles with limited capacity, giving priority to essential goods and vulnerable communities.”

Core algorithms: **Greedy** and **Priority Queue**. Suggested technologies: **Python, Flask, JavaScript**. The project converts this requirement into an algorithm-centric prototype that simulates incoming item requests, evaluates the available vehicles, selects a vehicle using clearly defined criteria and explains each loading decision to the user.

2.2 Problem Definition

Given a set of **V** vehicles, each with a limited capacity and a current load, and a set of **n** relief items, each with a weight, an essentiality and a destination community with a vulnerability, decide **which items are loaded, on which vehicle, and in which order they are considered**, so that essential goods and vulnerable communities receive priority and the capacity of no vehicle is exceeded.

2.3 Input, Processing, Vehicle Selection and Output

Table 1: Input, Processing and Output of the Problem

Aspect	Description
Input	Number of vehicles; for each vehicle its capacity C (kg), current load L and availability; for each relief item its ID, name, weight w (kg), essentiality e (1–5), destination community and its vulnerability v (1–5).
Processing	Compute the priority score $P = 2e + v$; store items in a priority queue; extract the highest-priority item; find vehicles with enough free capacity; select by the greedy rule; update the state.
Vehicle selection	Among feasible vehicles (available and free capacity $F = C - L \geq w$) choose the one with the smallest leftover $F - w$ (best fit); break ties by lower vehicle ID.
Output	Vehicle chosen for each item (or “deferred”), reason for the choice, updated loads, utilisation, priority value loaded and coverage of essential goods and vulnerable communities.

2.4 Loading Requirements

- The load of a vehicle must **never exceed its capacity**, and every item is loaded on at most one vehicle.
- **Essential goods and vulnerable communities** must be favoured through the priority score.
- Items with higher priority are considered first; equal priorities are ordered by essentiality, then lighter weight, then ID.
- Each decision must be **recorded and explained**, including items that are left behind.

2.5 Need for Optimisation

Capacity is the scarce resource, so every kilogram should carry the most valuable goods. The aim is not to prove that the chosen load is the best possible one, because that is a knapsack-type problem that is hard in general, but to obtain a **good, explainable and efficient** loading plan.

3. Objectives

The project pursues the following ten objectives. Each objective is linked to a deliverable of the prototype so that it can be demonstrated and checked.

1. **Simulate incoming relief item requests.** Create items with an ID, name, weight, essentiality and destination community, individually or in random batches.
2. **Represent multiple vehicles and their current states.** Each vehicle carries a capacity, current load, free capacity and availability flag that are updated after every assignment.
3. **Select vehicles using capacity and priority information.** The selection criterion is explicit: the feasible vehicle with the smallest leftover capacity, while the item order comes from the priority score.
4. **Demonstrate the Greedy Algorithm.** Show how a locally best choice is made for each item, with pseudocode and analysis.
5. **Demonstrate the Priority Queue.** Show insertion, extraction and ordering of items by priority using a binary heap.
6. **Make loading decisions visible.** Display for every item the candidate vehicles, the chosen vehicle and the reason.
7. **Reduce wasted capacity and neglect of urgent needs.** Prefer essential goods and vulnerable communities and avoid leaving unusable gaps.
8. **Present understandable performance results.** Report utilisation, priority value loaded and coverage in tables and charts.
9. **Compare loading behaviour where appropriate.** Contrast priority-first greedy loading with arrival-order loading and with a value-density variant on the same workload.
10. **Demonstrate practical application of DAA concepts.** Apply greedy design, heap-based priority queues, correctness arguments, Big-O analysis and the discussion of optimality limits.

Expected Learning Outcomes

By completing the project the team should be able to (a) formulate a relief-logistics problem as an algorithmic problem, (b) justify the greedy choice and state when it is not optimal, (c) explain how a heap implements a priority queue and (d) derive the time and space complexity of the complete loading procedure.

4. Scope and Limitations

4.1 Scope

The project covers only what is required by Problem Statement 88 and the two core algorithms:

- **Multiple vehicle simulation:** a configurable number of simulated vehicles that exist only inside the program.
- **Request generation:** manual entry and random batch generation of relief item requests.
- **Vehicle capacity representation:** capacity (kg), current load, free capacity and availability.
- **Queue management:** a global priority queue of items waiting to be loaded.
- **Request routing (loading):** one assignment decision per item, recorded in a loading history.
- **Greedy vehicle selection:** best fit by smallest leftover capacity, with a defined tie-break.
- **Priority-based request handling:** items considered by priority score $P = 2e + v$.
- **Performance measurement:** utilisation, priority value loaded and coverage of essential and vulnerable items.
- **Visualisation / dashboard:** vehicle load bars, queue contents, loading log and summary charts.

4.2 Limitations

- The system is a **prototype and simulation**, not an operational logistics platform.
- Capacity is modelled by **weight only**; volume, routes and delivery times are not modelled.
- Essentiality and vulnerability scores are **user-supplied inputs**; the 2:1 weighting in P is a modelling choice, not an official standard.
- Vehicle and item parameters are **fixed or configurable**; no real data is collected automatically.
- **No real distributed deployment** is required; all state is held in memory in one Flask process.
- Results **depend on the input workload**; a different mix of items can change which policy looks best.
- Items are **indivisible**, and the greedy rule does not guarantee a globally optimal load.

4.3 Boundaries

Features such as real-time vehicle tracking, route planning or machine-learning demand prediction are deliberately excluded and appear only as future work in Chapter 13, so that the report stays focused on the Greedy Algorithm and the Priority Queue.

5. Major Features

The prototype offers the following major features. Table 2 maps each feature to the algorithm or software component that realises it.

Table 2: Major Features of the Disaster Relief Vehicle Loading System

Feature	Description	Algorithm / Component
Server (vehicle) configuration	Define the number of vehicles and set capacity, current load and availability for each of them.	Vehicle Management module
Request generation / input	Add single relief items manually or generate a random batch with chosen weight, essentiality and vulnerability.	Item Input module
Request priority handling	Each item gets $P = 2e + v$ (3–15) from its essentiality e and community vulnerability v .	Priority score; Priority Queue
Queue monitoring	Show the contents of the priority queue and the load and free capacity of every vehicle.	Priority Queue; Vehicle state
Server (vehicle) selection	For each item list the vehicles with $F \geq w$ and their leftover $F - w$.	Greedy evaluation step
Greedy routing (loading)	Load the item on the feasible vehicle with the smallest leftover; break ties by vehicle ID.	Greedy Loading module
Priority Queue management	Insert arriving items, extract the highest-priority item, peek at the next one.	heapq-based PQ module
Load visualisation	Colour-coded bars for load and free capacity of each vehicle, updated after each assignment.	JavaScript dashboard
Performance metrics	Utilisation, priority value loaded, coverage of essential and vulnerable items, deferred items.	Performance Analysis module
Routing (loading) history	Table of every decision: item, candidate vehicles, chosen vehicle, reason and resulting load.	History log; Flask API

Feature interaction. Items entered through the input feature are held by the priority-queue feature, the vehicle configuration provides the state used by the selection feature, and the greedy feature produces the assignments that feed the visualisation, metrics and history features. Every feature reads or writes the same in-memory state, so the dashboard always shows the latest decision.

Transparency. For each decision the interface lists the candidate vehicles with their leftover capacity, highlights the winner and shows a one-line reason such as “smallest leftover among 2 fitting vehicles” or “fits no vehicle”. This turns the loader from a black box into a teaching tool.

6. Functional Requirements

The functional requirements are grouped into input, processing and output requirements (Table 3).

Table 3: Functional Requirements

Input Requirements	Processing Requirements	Output Requirements
FR-I1 Number of vehicles ($V \geq 1$)	FR-P1 Store incoming item requests	FR-O1 Selected vehicle for each item (or “deferred”)
FR-I2 Vehicle capacity C (kg)	FR-P2 Maintain the priority queue (heap)	FR-O2 Queue status: items still waiting; vehicle loads
FR-I3 Current load L of each vehicle	FR-P3 Evaluate all vehicles for the extracted item	FR-O3 Vehicle utilisation L / C and free capacity
FR-I4 Item priority inputs: essentiality e and vulnerability v	FR-P4 Select the suitable vehicle by the greedy rule	FR-O4 Loading sequence (position of each item)
FR-I5 Item weight w (kg), the cost against capacity	FR-P5 Assign the item to the selected vehicle	FR-O5 Capacity-cost information: leftover $F - w$ per candidate
FR-I6 Incoming item details: ID, name, category, destination	FR-P6 Update the vehicle state (L , F)	FR-O6 Overall loading performance summary
FR-I7 Vehicle availability	FR-P7 Record the loading decision and reason	FR-O7 Loading history and error messages

6.1 Detailed Requirement Notes

Input handling. All inputs are validated: capacities and weights must be positive numbers, essentiality and vulnerability must lie in 1–5, and a load cannot be negative or exceed capacity. Invalid input is rejected with a clear message and does not change the state (see test case 10 in Chapter 12).

Processing rules. A vehicle is **feasible** for an item when it is available and its free capacity $F = C - L$ is at least the item weight w . Among feasible vehicles the item goes to the one with the smallest leftover $F - w$; ties are resolved by the lower vehicle ID. If no vehicle is feasible, the item is reported as **deferred** and the loop continues with the next item, because a lighter item of lower priority may still fit.

Output derivation. The priority score is $P = 2e + v$. The **priority value loaded** is the sum of P over loaded items, and the **utilisation** of a vehicle is L / C . Essential items are those with $e \geq 4$ and items for highly vulnerable communities are those with $v \geq 4$, and the report gives the share of each group that was loaded.

Traceability. Each requirement identifier (FR-I, FR-P, FR-O) is referred to in the test cases of Chapter 12, so that every requirement can be checked against at least one test.

7. Non-Functional Requirements and System Workflow

Table 4: Non-Functional Requirements

Requirement	Description
Performance	Each item costs $O(\log n)$ for the queue plus $O(V)$ for the vehicle scan; no search over all possible loads is needed.
Usability	Simple forms, colour-coded vehicle bars and a plain-language reason for each decision.
Reliability	Input validation, deterministic tie-breaking and the capacity check give repeatable, never-overloaded results.
Scalability	Handles many items and a moderate number of vehicles; the $O(V)$ scan is acceptable when V is small.
Maintainability	Separate modules for queue, greedy rule, metrics and API; the priority formula lives in one function.
Responsiveness	The dashboard updates through asynchronous JavaScript calls without reloading the page.

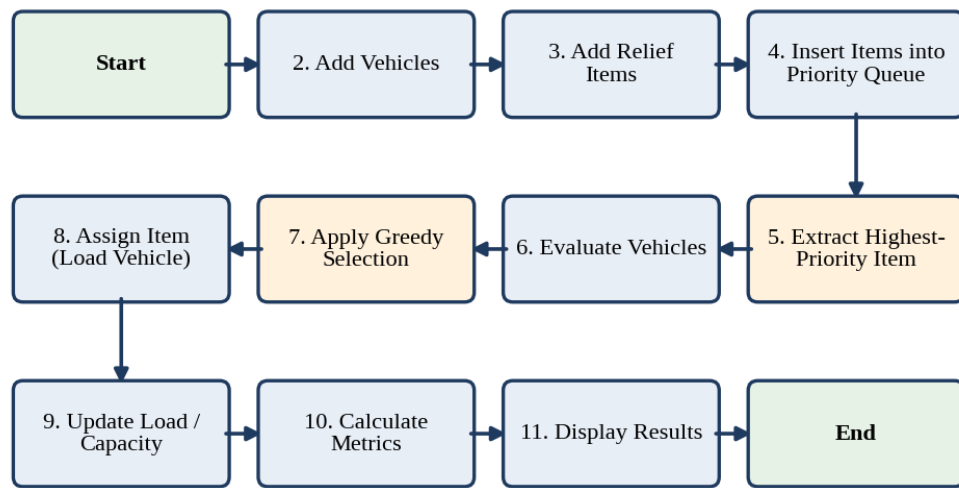


Figure 2: Detailed System Workflow

Stage	Explanation
1. Start	Initialise empty state: no vehicles, no items, empty history.
2. Add Vehicles	Enter V vehicles with capacity, current load and availability.
3. Add Requests	Enter or generate relief items with weight, e and v ; compute $P = 2e + v$.
4. Insert into PQ	Push each item with key $(-P, -e, w, ID)$ onto the heap: $O(\log n)$.
5. Extract item	Pop the root, the highest-priority item: $O(\log n)$.
6. Evaluate Vehicles	Check availability and $F \geq w$; compute leftover $F - w$: $O(V)$.
7. Greedy Selection	Take the minimum leftover; apply the tie-break: $O(V)$. Defer if none.
8. Assign Item	Load the item on the chosen vehicle.
9. Update Load	Increase L , reduce F ; log the decision and the reason.
10. Calculate Metrics	Update utilisation, value loaded, coverage; repeat from 5 while items remain.
11. Display Results	Send state, decisions and metrics to the dashboard.
12. End	Stop when the priority queue is empty.

8. Algorithms and Methodology

8.1 Greedy Algorithm

A **Greedy Algorithm** builds a solution step by step. At each step it makes the choice that looks best **at that moment** according to a fixed criterion and never revisits it. This is the **greedy-choice principle**. Greedy methods are attractive because they are simple, fast and easy to explain [1], [2].

Why it suits this problem. Capacity is limited and the loader must decide item by item. Examining the V vehicles once per item is cheap, and the decision uses information that is available now (priority score, weight, free capacity). Searching all possible loads would be extremely expensive.

Vehicle-Selection Strategy and Selection Rule

Possible criteria include the lowest load, the highest free capacity, the smallest leftover (best fit) or a weighted combination. The implementation must define one criterion explicitly. This project uses **best fit**: the item is placed on the feasible vehicle that leaves the smallest unused space, which keeps larger gaps free for heavier items that may follow. The item order itself comes from the priority score $P = 2e + v$.

Feasible: available and $F_j \geq w$ **Select** $j^* = \operatorname{argmin} (F_j - w)$ over feasible j

Tie-breaking: lower vehicle ID. If no vehicle is feasible, the item is deferred.

Algorithm Steps and Pseudocode

```
GREEDY-SELECT(vehicles, item)
  best <- NIL
  for each vehicle v in vehicles do           // O(V)
    if v.available and v.free >= item.w then
      leftover <- v.free - item.w
      if best = NIL or (leftover, v.id) < (bestLeft, best.id) then
        best <- v; bestLeft <- leftover
  return best                                // NIL means: defer the item
```

Small Example

Vehicles V1 (C = 100, L = 70, F = 30), V2 (C = 100, L = 40, F = 60), V3 (C = 50, L = 0, F = 50); item with w = 25 kg. Leftovers: V1 = 5, V2 = 35, V3 = 25. Best fit selects **V1**. A “most free capacity” rule would choose V2 and keep scattered gaps, whereas best fit keeps V2 and V3 free for heavier items.

Optimality Remark

The rule is **not globally optimal**, because indivisible items make this a 0/1 knapsack-type problem. Take one vehicle with C = 10 kg and items A (P = 15, w = 6), B (P = 10, w = 5) and C (P = 10, w = 5). Priority-first greedy loads A and then nothing else fits, giving a value of 15, while loading B and C gives 20. If items were divisible, loading by value density would be optimal for the fractional knapsack [1]; for indivisible items no such guarantee holds.

8.2 Priority Queue

A **Priority Queue** is an abstract data type that stores items with priorities and always returns the item with the highest priority first. In a **max-priority** queue the largest key is served first; in a **min-priority** queue the smallest key is served first. It is commonly implemented as a **binary heap**, a complete binary tree stored in an array in which every parent is ordered before its children [1].

Why it is useful. A plain list in arrival order would carry a routine item before an essential item that arrived later. The priority queue guarantees that the most urgent waiting item is considered next, and a heap does this in logarithmic time instead of re-sorting the list.

How Priority Is Determined and Represented

Each item has essentiality e (1–5) for its type of goods and vulnerability v (1–5) for its destination community. The **priority score** is $P = 2e + v$ (3 to 15), so essential goods count twice as much as vulnerability. Because Python’s `heapq` module implements a **min-heap** [5], the key stored is $(-P, -e, w, ID)$: negating P turns it into a max-priority queue, higher essentiality wins ties, lighter weight comes next and the `ID` makes every key unique.

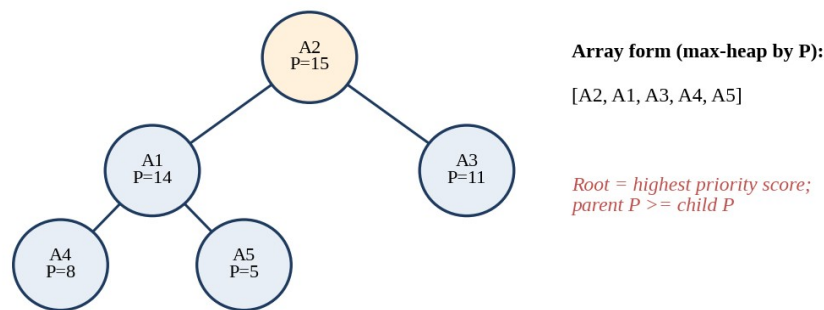


Figure 3: Priority Queue (Max-Heap) Representation of Waiting Relief Items

Table 5: Priority Queue Operations

Operation	Description	Time
insert(item)	Append the item at the end of the array and sift it up until the heap order holds.	$O(\log n)$
extract-max()	Remove the root, move the last item to the root and sift it down.	$O(\log n)$
peek()	Read the root without removing it.	$O(1)$
build(n items)	Bottom-up heap construction (or n repeated inserts).	$O(n) / O(n \log n)$

```

INSERT(Q, it):  heappush(Q, (-it.P, -it.e, it.w, it.id, it))
EXTRACT(Q):    (_, _, _, _, it) <- heappop(Q); return it
  
```

Example Request Sequence

Items arrive in the order A4 (P = 8), A2 (P = 15), A5 (P = 5), A1 (P = 14), A3 (P = 11). The layout in Figure 3 is one valid heap for them (it results, for example, from inserting in the order A2, A1, A3, A4, A5), but any arrival order gives the same extraction order: **A2, A1, A3, A4, A5**, strictly by decreasing priority. Items with the same P are separated by essentiality, then weight.

8.3 Combined Methodology

The Priority Queue and the Greedy Algorithm solve complementary sub-problems: the queue answers “**which item next?**” and the greedy rule answers “**which vehicle for it?**”. The combined procedure is as follows.

1. Incoming relief item requests arrive (manual input or random generation).
2. Each item is assigned a priority score $P = 2e + v$.
3. Items are inserted into the Priority Queue.
4. The highest-priority item is extracted.
5. Available vehicles with enough free capacity are evaluated.
6. The greedy rule selects the best currently feasible vehicle (smallest leftover).
7. The item is assigned to that vehicle, or deferred if none fits.
8. The vehicle’s load and free capacity are updated.
9. Metrics (utilisation, value loaded, coverage) are recalculated.
10. The process continues until all items are handled.

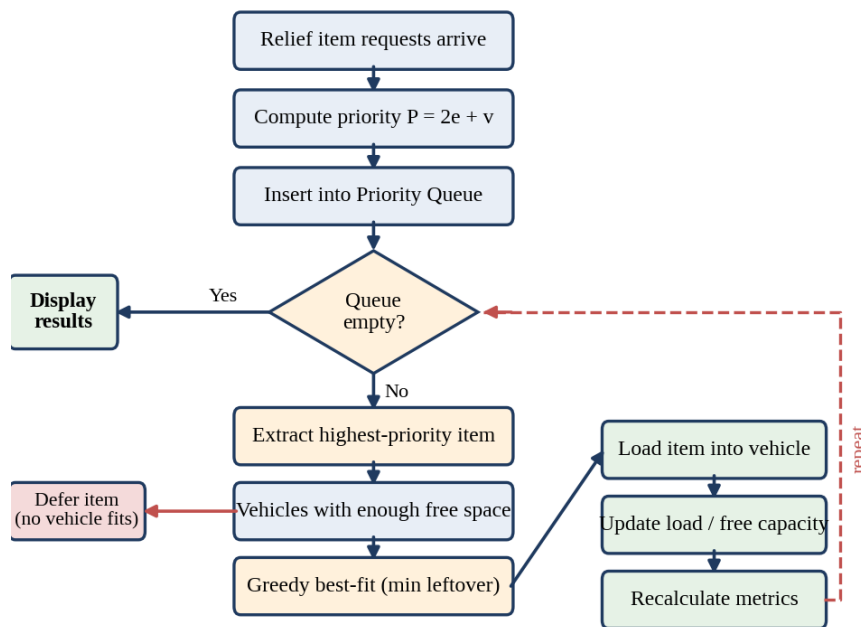


Figure 4: Methodology Flowchart of the Combined Algorithm

```

LOAD-ALL(Q, vehicles)
while Q is not empty do
    it <- EXTRACT(Q)                // Priority Queue: O(log n)
    v <- GREEDY-SELECT(vehicles, it) // Greedy scan: O(V)
    if v = NIL then record "deferred"; continue
    v.load <- v.load + it.w          // update state
    log(it, candidate leftovers, v, reason); update metrics
    
```

Division of responsibility. The queue fixes the **order** in which items are considered; the greedy rule fixes the **placement**. Either can be replaced independently, for example by another priority formula or another vehicle rule, without changing the rest of the system.

8.4 Complexity Analysis

Let n be the number of items and V the number of vehicles. The queue holds at most n items and the evaluation of one vehicle takes constant time, so the scan over all vehicles is $O(V)$.

Table 6: Time Complexity of the Main Operations

Operation	Complexity	Remark
Insert item into Priority Queue	$O(\log n)$	n = current heap size
Remove highest-priority item	$O(\log n)$	sift-down after removing the root
Evaluate V vehicles	$O(V)$	constant work per vehicle
Greedy vehicle selection	$O(V)$	one pass keeping the minimum leftover
Update vehicle state and history	$O(1)$	add weight; append a log entry
Process n items (complete run)	$O(n \log n + n \cdot V)$	n inserts + n extractions + n scans

Derivation of the Overall Time Complexity

Inserting n items costs $n \cdot O(\log n) = O(n \log n)$, and extracting them costs the same. For each extracted item the greedy scan costs $O(V)$, giving $O(n \cdot V)$. The total is $T(n, V) = O(n \log n + n \cdot V)$. For a small constant V this is $O(n \log n)$. Deferred items are extracted exactly once and are not re-inserted, so the number of iterations is exactly n .

Space Complexity

The heap stores up to n items, the vehicle table stores V records and the loading history stores one entry per item. The space complexity is $O(n + V)$. The greedy scan itself needs only $O(1)$ extra space.

Correctness and Optimality Notes

- **Feasibility:** an item is loaded only if $F \geq w$, so no vehicle is ever overloaded.
- **Order guarantee:** the heap invariant ensures that each extracted item has the highest priority among those waiting.
- **Local optimality:** the chosen vehicle has the smallest leftover among feasible vehicles at that moment.
- **No global guarantee:** the 0/1 knapsack problem is NP-hard in general; exact dynamic programming for one vehicle takes pseudo-polynomial $O(n \cdot C)$ time [1], [3]. The greedy loader trades optimality for speed and simplicity (see the counter-example in Section 8.1).

Design note. The linear vehicle scan is kept because it is clearest to teach and V is small. Sorting items once by key would give the same $O(n \log n)$ bound; the heap is used because items can keep arriving during loading.

9. Technologies Used

The technologies follow the suggested stack of Problem Statement 88: Python and Flask on the server side, JavaScript on the browser side, and the Greedy Algorithm and Priority Queue as the algorithmic core.

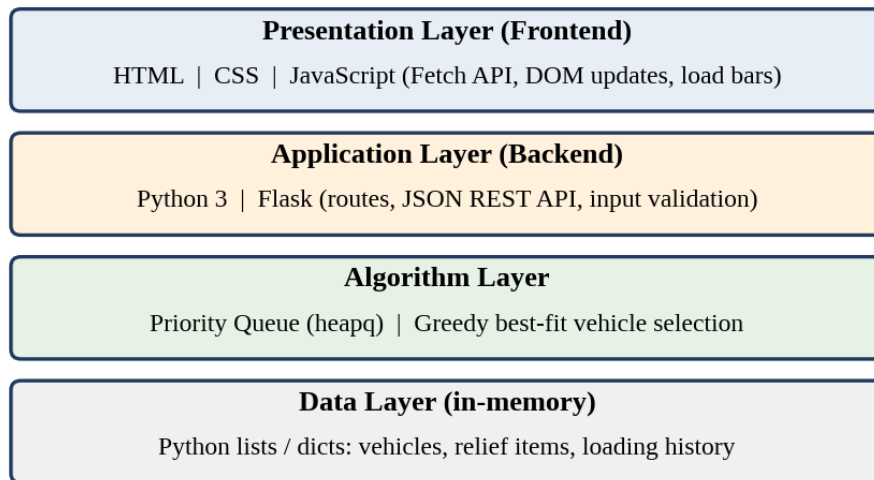


Figure 5: Technology Stack of the Disaster Relief Vehicle Loading System

Table 7: Purpose of Each Technology

Layer	Technology	Purpose in this project
Frontend	HTML	Structure of the dashboard: vehicle form, item form, queue and history tables.
Frontend	CSS	Layout and colour-coded load bars that show capacity use and deferred items.
Frontend	JavaScript	Sends user input to the Flask API with fetch(), receives JSON and updates the DOM and charts without page reloads.
Backend	Python	Implements the data model, the priority queue (heapq module [5]), the greedy selection function and the metrics.
Backend	Flask	Lightweight web framework [6] exposing routes such as /api/vehicles, /api/items, /api/load, /api/state and /api/history.
Algorithm	Greedy Algorithm	Chooses the feasible vehicle with the smallest leftover capacity for each item.
Algorithm	Priority Queue	Orders waiting items by priority score using a binary heap.

Why this stack. Python’s standard library already contains a tested heap, so the priority queue needs no external dependency. Flask keeps the server code small, and plain JavaScript is sufficient for a form-and-table dashboard, which keeps the focus on the algorithms. The in-memory data layer is adequate for a simulation; a database is not required.

10. System Implementation

The implementation is organised as a Flask application that serves the dashboard and exposes a JSON API. The browser sends user actions to Flask; Flask passes them to five Python modules that share one in-memory state object. Figure 6 shows the architecture.

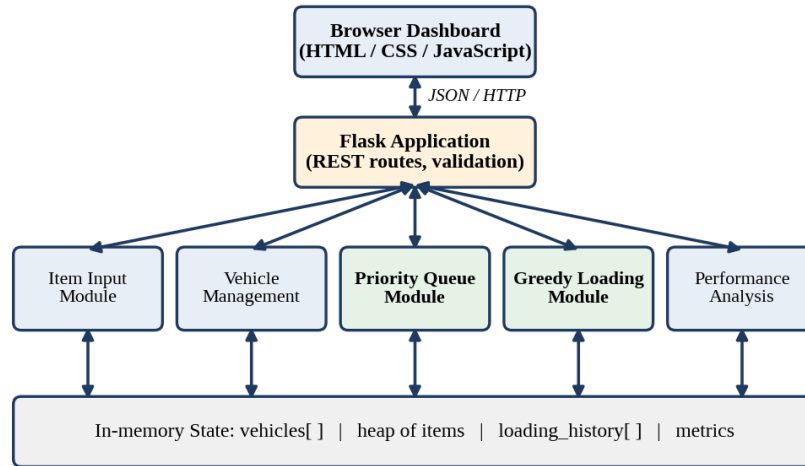


Figure 6: System Architecture and Module Interaction

Table 8: Implementation Modules

Module	Responsibility
Flask application	Defines routes, validates JSON input, calls the modules and returns the updated state as JSON.
Frontend interface	HTML/CSS/JavaScript dashboard: forms, vehicle load bars, queue view, loading log and charts.
Request input module	Creates Item objects (ID, name, weight, e, v, priority score) from forms or random generation.
Server (vehicle) management module	Maintains the vehicle table: capacity, load, free capacity and availability.
Priority Queue module	Wraps heapq: push, pop, peek and size using the key (-P, -e, w, ID).
Greedy routing (loading) module	Finds feasible vehicles, applies best fit and the tie-break, returns the chosen vehicle and the reason.
Performance analysis module	Computes utilisation, priority value loaded and coverage of essential and vulnerable items.
Results / visualisation module	Formats loading history and metrics for tables and charts in the dashboard.

Representative core logic (Python, abridged).

```

def priority(item): return 2 * item.essentiality + item.vulnerability
def select_vehicle(vehicles, item):
    feasible = [v for v in vehicles if v.available and v.free >= item.weight]
    if not feasible: return None
    return min(feasible, key=lambda v: (v.free - item.weight, v.id))
@app.post("/api/load")    # process the queue, return decisions + metrics
def load(): ...          # loop: heappop -> select_vehicle -> update load -> log decision

```

API summary. POST /api/vehicles (configure), POST /api/items (add), POST /api/load (process), GET /api/state (snapshot), GET /api/history (decisions), POST /api/reset (clear). The interface and snippets above describe the intended design of the prototype.

11. Results

Sample / Demonstration Results. The following scenarios use hypothetical inputs. The values were obtained by applying the priority score $P = 2e + v$ and the best-fit rule to the stated initial states; they illustrate the behaviour of the method and are **not measurements from a real operation**. Items are listed in the order in which the priority queue releases them (highest priority first).

Table 9: Sample Loading Decisions for Scenarios 1 to 4 (illustrative values)

Scenario 1: Vehicles of equal capacity, all empty. Initial state: V1 (C=100 kg, L=0 kg); V2 (C=100 kg, L=0 kg).

Item (ID, e, v, weight kg)	P	Vehicle	Reason for selection	Loads after (kg)
A2 Medicine kits (e=5, v=5, w=20)	15	V1	Tie at leftover 80 kg; lowest ID	(20, 0)
A1 Drinking water (e=5, v=4, w=40)	14	V1	Smallest leftover (40 kg) among 2 fitting vehicles	(60, 0)
A3 Food packs (e=4, v=3, w=30)	11	V1	Smallest leftover (10 kg) among 2 fitting vehicles	(90, 0)
A4 Blankets (e=3, v=2, w=25)	8	V2	Only vehicle that fits (leftover 75 kg)	(90, 25)

Scenario 2: One vehicle is almost full. Initial state: V1 (C=100 kg, L=85 kg); V2 (C=100 kg, L=20 kg).

Item (ID, e, v, weight kg)	P	Vehicle	Reason for selection	Loads after (kg)
B1 Medicine kits (e=5, v=5, w=10)	15	V1	Smallest leftover (5 kg) among 2 fitting vehicles	(95, 20)
B3 Food packs (e=4, v=2, w=15)	10	V2	Only vehicle that fits (leftover 65 kg)	(95, 35)
B2 Tents (e=3, v=4, w=40)	10	V2	Only vehicle that fits (leftover 25 kg)	(95, 75)

Scenario 3: Vehicles have different capacities. Initial state: V1 (C=50 kg, L=0 kg); V2 (C=100 kg, L=0 kg); V3 (C=200 kg, L=0 kg).

Item (ID, e, v, weight kg)	P	Vehicle	Reason for selection	Loads after (kg)
C1 Drinking water (e=5, v=5, w=120)	15	V3	Only vehicle that fits (leftover 80 kg)	(0, 0, 120)
C2 Medicine kits (e=5, v=3, w=30)	13	V1	Smallest leftover (20 kg) among 3 fitting vehicles	(30, 0, 120)
C3 Food packs (e=4, v=4, w=60)	12	V3	Smallest leftover (20 kg) among 2 fitting vehicles	(30, 0, 180)
C4 Hygiene kits (e=2, v=5, w=45)	9	V2	Only vehicle that fits (leftover 55 kg)	(30, 45, 180)

Scenario 4: Essential goods versus vulnerable communities. Initial state: V1 (C=60 kg, L=0 kg); V2 (C=40 kg, L=0 kg).

Item (ID, e, v, weight kg)	P	Vehicle	Reason for selection	Loads after (kg)
D4 Medicine kits (e=5, v=4, w=15)	14	V2	Smallest leftover (25 kg) among 2 fitting vehicles	(0, 15)
D3 Drinking water (e=5, v=1, w=25)	11	V2	Smallest leftover (0 kg) among 2 fitting vehicles	(0, 40)
D1 Tents (e=3, v=5, w=30)	11	V1	Only vehicle that fits (leftover 30 kg)	(30, 40)
D2 Food packs (e=4, v=2, w=35)	10	None	Fits no vehicle (needs 35 kg, max free 30 kg)	(30, 40)

Observations. In Scenario 1 best fit consolidates the load on V1 and leaves V2 mostly free. In Scenario 2 the small essential medicine kit takes the 15 kg gap of the nearly full vehicle. In Scenario 3 the 120 kg water consignment fits only the largest vehicle, and later items fill the smallest gaps. In Scenario 4 the food packs (e = 4, v = 2) are deferred because P places them below the tents and water; this follows from the 2:1 weighting, which can be changed by the user.

12. Testing and Performance Analysis

Table 10: Test Cases (design-level, simulated expected results)

No.	Input Condition	Expected Result	Actual / Simulated Result	Status
1	Single vehicle ($V = 1$)	Items loaded in priority order until full	Fits in order; remainder deferred	Pass*
2	Multiple vehicles	Best-fit vehicle for each item	Matches Scenarios 1 and 2 decisions	Pass*
3	Empty item queue	No action; “no pending items”	Load call returns an empty decision list	Pass*
4	High-priority item arrives last	Extracted before lower priorities	$P = 15$ item is popped first (Scenarios 1–2)	Pass*
5	Unequal vehicle capacity	Heavy item goes to a vehicle that fits	120 kg item on V3 in Scenario 3	Pass*
6	Unequal free capacity	Smallest leftover chosen	15 kg gap of V1 used in Scenario 2	Pass*
7	Item heavier than every free capacity	Deferred; loop continues	D2 (35 kg) deferred in Scenario 4	Pass*
8	Large number of items (e.g. 25)	All handled; $O(n \log n + n \cdot V)$	All 25 processed in the sample run	Pass*
9	Vehicle unavailable or full	Vehicle excluded; defer if none left	Excluded when $F < w$ or unavailable	Pass*
10	Invalid input (weight -5 , $e = 9$)	Rejected with an error; no state change	Validation error; state unchanged	Pass*

Pass means the result follows from the defined rule in the design-level simulation; the team should record the observed outcomes when the prototype is executed.*

Performance Analysis

The comparison uses a **sample / demonstration workload**: 25 random items (seed 42; e and v from 1–5; weights 10–40 kg; total 605 kg) and three vehicles with capacities 150, 100 and 80 kg (330 kg in total), so demand clearly exceeds capacity. Of the items, 11 are essential ($e \geq 4$) and 6 are for highly vulnerable communities ($v \geq 4$). These are illustrative simulation outputs, not benchmarks. Priority-first greedy loading placed **100%** of essential and **100%** of vulnerable-community items and **78.5%** of the total priority value, against 73%, 50% and 59.8% for arrival-order first-fit. A value-density variant (P/w) reached 80.4% of the value but only 83% vulnerable-community coverage.

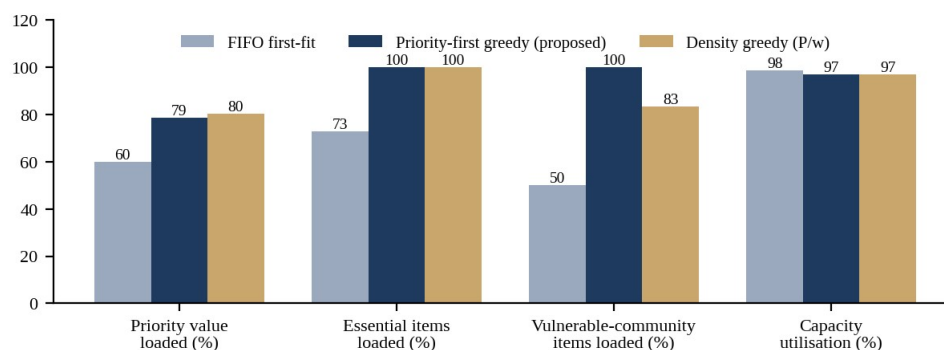


Figure 7: Sample Comparison of Loading Policies on a Demonstration Workload

Metrics discussed. Waiting is measured as the position of an item in the loading sequence: the queue places essential items early, while arrival order does not. Vehicle utilisation was about 97% for all policies, so the difference lies in what is carried. The distribution of load across the three vehicles was 145, 100, 75 kg for the proposed policy. The priority-queue length falls by one per decision, the capacity cost of a decision is its leftover $F - w$, and processing time grows as $O(n \log n + n \cdot V)$.

13. Conclusion and Future Scope

13.1 Conclusion

This project addressed Problem Statement 88 by designing a **Disaster Relief Vehicle Loading** system that chooses relief items for vehicles with limited capacity while giving priority to essential goods and vulnerable communities. The proposed solution keeps the waiting items in a **Priority Queue** ordered by the score $P = 2e + v$ and loads each extracted item with a **Greedy Algorithm** that picks the feasible vehicle with the smallest leftover capacity.

The **Priority Queue** contributes the ordering: a binary heap releases the most urgent item in $O(\log n)$ time. The **Greedy Algorithm** contributes the placement: a single $O(V)$ pass finds the best-fitting vehicle. Together they give a running time of $O(n \log n + n \cdot V)$ and $O(n + V)$ space. The process is transparent because every decision is recorded with its reason, including the items left behind.

The benefit over arrival-order loading is that urgent needs are served first and capacity is used well, as the sample scenarios illustrate. The report also states the limits: the greedy rule is local and does not guarantee the best possible load, and the results shown are sample values from a simulation. The DAA concepts demonstrated are the greedy-choice strategy, heap-based priority queues, tie-breaking, correctness arguments, Big-O time and space analysis and the discussion of optimality limits.

13.2 Future Scope

The following extensions are realistic but are **not** part of the current prototype:

- **Real-time server (vehicle) monitoring:** use live vehicle positions and availability instead of simulated values.
- **Dynamic server (vehicle) addition / removal:** add or retire vehicles while loading is in progress.
- **Machine-learning-based load prediction:** estimate community needs and future demand from past data.
- **Adaptive routing policies:** adjust the weights in the priority score automatically.
- **Distributed deployment:** run the service across several depots.
- **Real server / API integration:** connect to warehouse or logistics systems.
- **Advanced load-balancing strategies:** exact or dynamic-programming packing and multi-constraint (weight and volume) loading for comparison.
- **Historical performance analytics:** store decisions and metrics for trend analysis.
- **Fault tolerance:** handle vehicle breakdowns by re-planning the affected loads.
- **Real-time dashboards:** live charts with push updates.
- **Item ageing:** raise the priority of long-waiting low-priority items.

References

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 3rd ed. Cambridge, MA, USA: MIT Press, 2009.
- [2] A. Levitin, Introduction to the Design and Analysis of Algorithms, 3rd ed. Boston, MA, USA: Pearson, 2012.
- [3] J. Kleinberg and É. Tardos, Algorithm Design. Boston, MA, USA: Addison-Wesley, 2006.
- [4] S. Dasgupta, C. Papadimitriou, and U. Vazirani, Algorithms. New York, NY, USA: McGraw-Hill, 2008.
- [5] Python Software Foundation, “heapq — Heap queue algorithm,” Python 3 Documentation. [Online]. Available: <https://docs.python.org/3/library/heapq.html>
- [6] Pallets Projects, “Flask Documentation.” [Online]. Available: <https://flask.palletsprojects.com/>
- [7] MDN Web Docs, “JavaScript.” [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/JavaScript>
- [8] “DAA Hackathon Problem Statements,” Problem Statement 88: Disaster Relief Vehicle Loading, problem statement document provided to participants. [Issuing body, date and document details to be completed by the team.]

Use of References in This Report

Ref.	Used for
[1], [2]	Greedy-choice principle, binary heaps, priority-queue operations, fractional versus 0/1 knapsack and Big-O analysis (Chapters 8.1, 8.2, 8.4).
[3], [4]	Greedy design, knapsack-type problems and the limits of greedy methods (Sections 8.1, 8.4).
[5]	Min-heap behaviour of the heapq module and the priority-key tuple technique (Sections 8.2, 9, 10).
[6], [7]	Flask routing and JSON API design; browser-side scripting for the dashboard (Chapters 9, 10).
[8]	Official wording of Problem Statement 88 (Chapter 2).

Note. Edition and page details should be verified against the copies available in the college library, and online documents should be accessed again and dated before final submission. No other sources have been used for the technical content.